

Tab 1

This is an ambitious project. Let me give you a grounded, phase-by-phase timeline assuming you're a solo developer using Claude Code as your primary coding partner.

First, a key reframe: Don't try to build the full vision on Day 1. Ship a narrow, usable slice first, then layer on complexity. Here's how I'd break it down:

Phase 0 — Foundation & Design (Week 1–2)

Nail down your tech stack, set up the repo, CI/CD, and design the database schema. Claude Code is excellent here — it can scaffold projects, write configs, and generate boilerplate fast. Decide on your stack early: something like Next.js + Python/FastAPI backend + Postgres (start simple, Neo4j can come later) + Anthropic API for the Socratic engine.

Phase 1 — The "Talkative Compiler" MVP (Weeks 3–8, ~6 weeks)

This is your core differentiator, so spend the most time here. Build just enough to demo one algorithm topic (say, Binary Search) end-to-end:

- A code editor UI (Monaco/CodeMirror) with the locked Run button — ~1 week
- The "Pre-Flight Check" Socratic dialogue using Claude API — ~2 weeks (prompt engineering is the hard part, not the integration)
- The "Adversarial Debugger" flow (run code in a sandbox, capture errors, feed to LLM for Socratic hints instead of raw output) — ~2 weeks
- Basic auth, progress tracking, a handful of problems — ~1 week

Claude Code will dramatically speed up the frontend and API plumbing. The time sink will be **prompt engineering** — crafting prompts that produce genuinely good Socratic questions rather than generic ones. Budget real time for that iteration loop.

Phase 2 — Content & Course Structure (Weeks 9–12, ~4 weeks)

- Build out 15–20 algorithm problems across 3–4 topics
 - Implement the concept dependency graph (start with a simple DAG in Postgres using adjacency lists, not Neo4j — you don't need a graph DB yet)
 - Add the "Logic Gate" quality checks per problem
 - Basic learner profile tracking (which concepts mastered, which weak)
-

Phase 3 — Polish & Deploy Beta (Weeks 13–16, ~4 weeks)

- Code sandbox hardening (security is critical if you're running user code — use something like Judge0 or Piston)

- UI polish, mobile responsiveness
 - Deploy (Vercel/Railway + managed DB)
 - Get 5–10 beta users and iterate on the Socratic prompts based on real conversations
-

Phase 4 — The "Hard Mode" HFT Track & Expansion (Weeks 17–24, ~8 weeks)

- C++ track with cache-locality gates
 - The visual memory map / stack-heap debugger (this is a significant frontend effort)
 - Multi-course structure, bridge modules
 - Learning telemetry dashboard
-

Realistic totals:

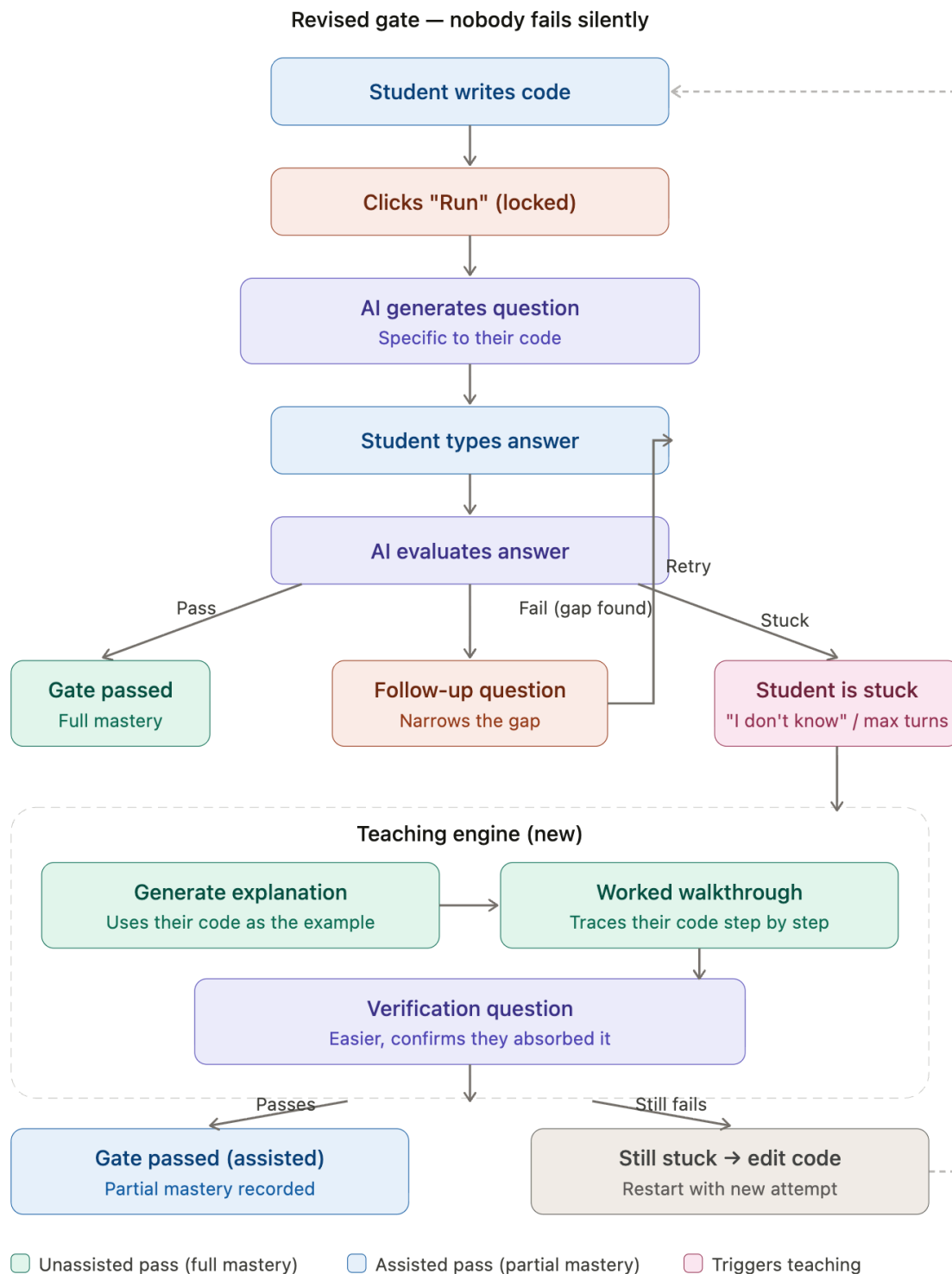
Milestone	Timeline	What you ship
Usable MVP (1 topic, Socratic flow works)	~8 weeks	Demo-able product
Beta with real content	~16 weeks (~4 months)	Invite beta users
Full Phase 1 course + HFT track	~24 weeks (~6 months)	Public launch candidate
Multi-course platform (Phases 2–4 of your roadmap)	12–18 months	The full Logos vision

Where Claude Code saves you the most time: scaffolding, CRUD, API routes, frontend components, test generation, and debugging. Where it *won't* save you time: prompt engineering for the Socratic engine, UX decisions, and the pedagogical design of what makes a good "Logic Gate" question. That's your secret sauce and it requires human iteration.

My honest advice: Treat the 8-week MVP as your only goal right now. One topic, five problems, the Socratic flow working well enough to impress someone in a demo. Everything else is Phase 2. Would you like me to help you sketch out the tech stack or scaffold anything to get started?

Design Course Wise

This is the main loop of the app for different courses also the same loop will be followed



When does teaching trigger? Three signals, any of which fires it:

1. The student explicitly says "I don't know" or "I'm stuck" — the evaluator classifies this as a **STUCK** verdict (not **FAIL**). A fail means they tried but got it wrong. Stuck means they need teaching.
2. Max turns exhausted (4 follow-up questions, all failed) — at this point, narrower questions aren't helping. The gap is foundational.
3. The evaluator detects the student is *completely* lost — their answer reveals a fundamental misconception (e.g., they think binary search works on unsorted arrays). This is a new **STUCK** verdict from the evaluator, distinct from **FAIL**.

What does the teaching response contain? Three layers, delivered as one streamed response:

1. A targeted concept explanation — not a textbook definition, but an explanation *grounded in their specific code*. "In your code, **left** and **right** define the search window. Here's what happens when..."
2. A worked walkthrough — trace their actual code with a concrete input, step by step, showing what each variable holds. This is what students *should* be doing in their heads.
3. A verification question — easier than the original, designed to confirm they absorbed the teaching. "Now that you've seen the trace, what would **left** be after the first iteration if **nums** = [1, 3, 5, 7] and **target** = 5?"

Learner profile

Learner profile — five layers

Static at top, dynamic at bottom

Layer 1: Identity

Track (HFT/general), language (Python/C++), goal, signup date

Set once

Layer 2: Concept mastery graph

Per-concept scores: binary_search: 1.0, recursion: 0.5, hash_maps: 0.0

Includes assist_level: unassisted / assisted / untested

Updated per session

Layer 3: Mistake patterns

Recurring error tags: "off_by_one": 4 times, "missing_base_case": 2 times

Weakness clusters derived from failed gate sessions

Computed nightly

Layer 4: Recent session context

Last 3 sessions: questions asked, answers given, pass/fail

What they were taught (if teaching triggered)

Rolling window

Layer 5: Learning velocity

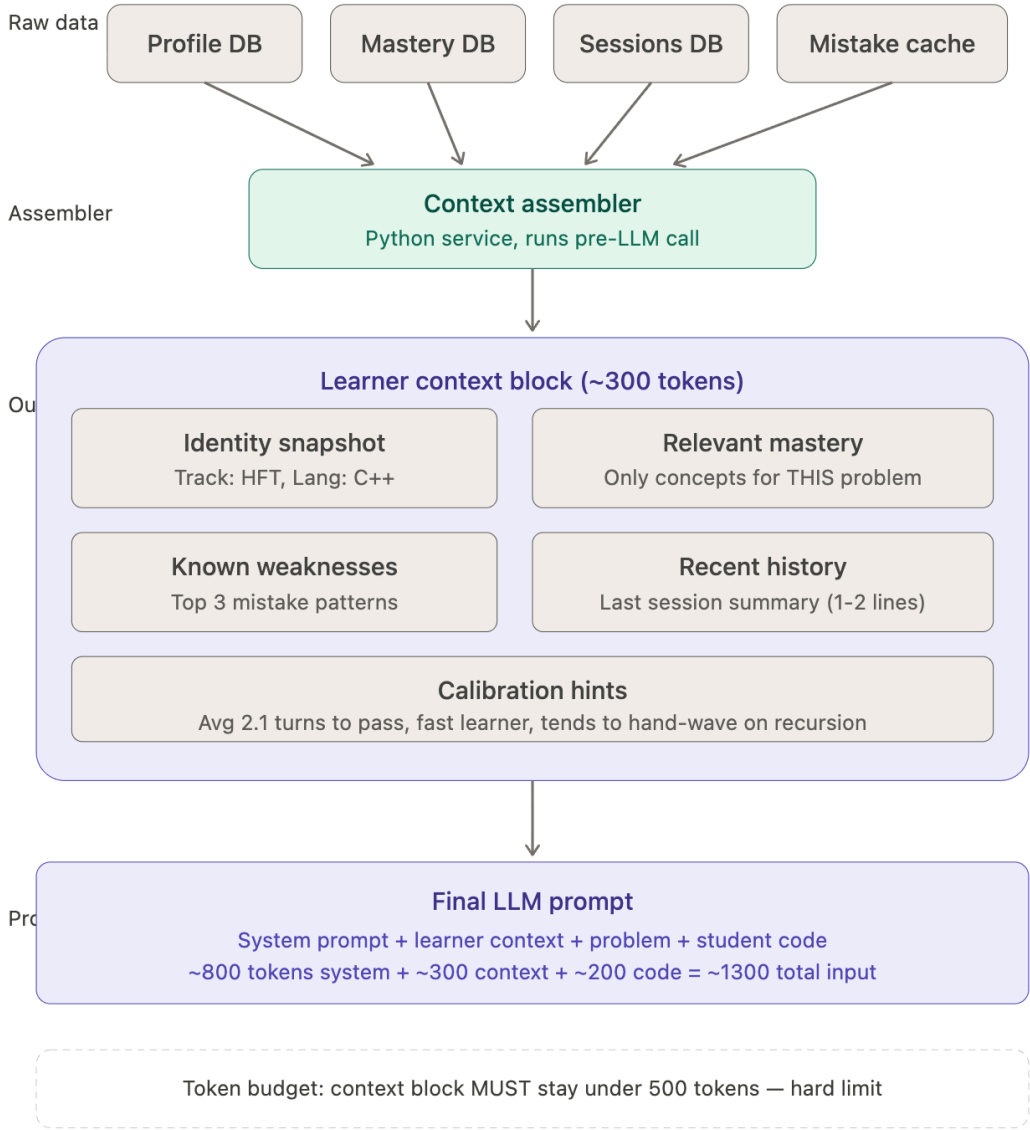
Avg turns to pass gate, time-to-mastery per concept

Calibrates question difficulty + teaching depth

Computed weekly

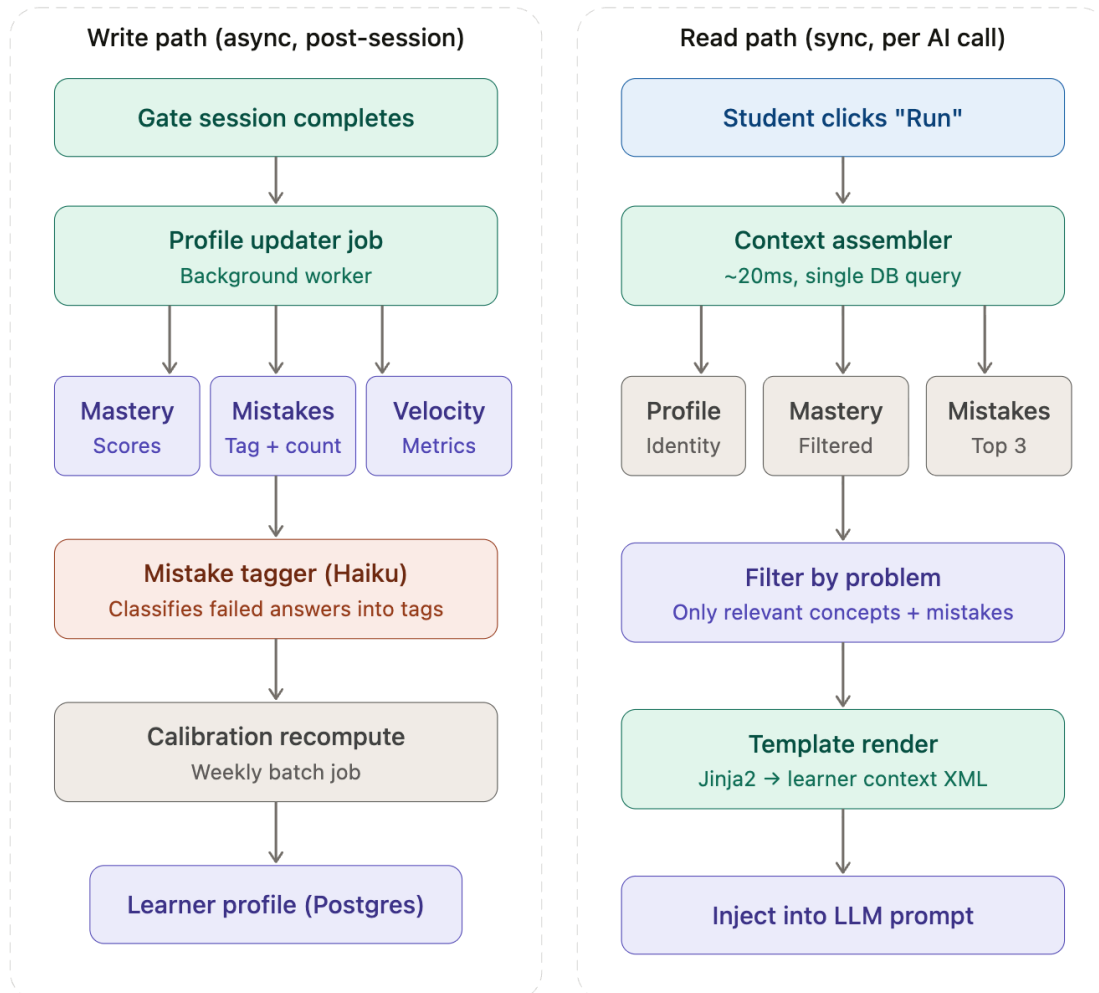
Now the critical engineering question — how all five layers get compressed into a compact context that's injected into every LLM call. You can't send the full profile (that'd be thousands of tokens). You need a "context distillation" pipeline:

Context assembly — what the AI actually sees



Now the data flow — how this profile gets built and updated. There are two paths: the write path (data flowing in after each session) and the read path (context assembled before each AI call):

Two paths — write after session, read before AI call



HLD

Logos — system architecture

